Paradigma OOP

Description	Paradigma per organizzare il codice intorno agli oggetti

La programmazione orientata agli oggetti (OOP) è un paradigma di programmazione che consente di organizzare il codice intorno agli oggetti, ovvero istanze di classi. Le classi sono modelli per creare oggetti, e gli oggetti sono istanze di queste classi.

L'OOP in Python include concetti fondamentali come classi, oggetti, ereditarietà, incapsulamento e polimorfismo.

Classe:

Una classe è un modello per creare oggetti. Definisce le proprietà (attributi) e i comportamenti (metodi) che gli oggetti di quella classe avranno.

```
pythonCopy code
class Car:
    def __init__(self, brand, model):
        self.brand = brand
        self.model = model

    def display_info(self):
        print(f"{self.brand} {self.model}")
```

Oggetto:

Un oggetto è un'istanza di una classe. Può avere attributi che rappresentano le sue caratteristiche e metodi che definiscono il suo comportamento.

```
pythonCopy code
my_car = Car(brand="Toyota", model="Camry")
```

Costruttore (`__init__`):

Il costruttore è un metodo speciale chiamato quando un nuovo oggetto viene creato.
In Python, è comunemente chiamato `__init__` e viene utilizzato per inizializzare gli attributi dell'oggetto.

```
pythonCopy code
def __init__(self, brand, model):
    self.brand = brand
    self.model = model
```

Ereditarietà:

La ereditarietà consente di creare una nuova classe basata su una classe esistente. La nuova classe eredita gli attributi e i metodi della classe genitore.

```
pythonCopy code
class ElectricCar(Car):
    def __init__(self, brand, model, battery_capacity):
        super().__init__(brand, model)
        self.battery_capacity = battery_capacity
```

Incapsulamento:

L'incapsulamento consiste nel nascondere i dettagli di implementazione di una classe e consentire l'accesso solo attraverso metodi pubblici.
In Python, l'incapsulamento può essere implementato con attributi e metodi privati (che iniziano con due trattini bassi `__`).

```
pythonCopy code
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance

    def get_balance(self):
        return self.__balance
```

Polimorfismo:

Il polimorfismo consente a oggetti di classi diverse di rispondere allo stesso metodo in modo diverso. In Python, il polimorfismo è spesso implementato attraverso l'uso di metodi con lo stesso nome in classi diverse.

```
pythonCopy code
class Cat:
    def make_sound(self):
        print("Meow!")

class Dog:
    def make_sound(self):
        print("Woof!")
```

Variabili Statiche:

```
pythonCopy code
class MiaClasse:
```

```
    variabile_statica = 0

    def __init__(self, valore):
        self.valore = valore
        MiaClasse.variabile_statica += 1

# Accesso alla variabile statica
print(MiaClasse.variabile_statica)  # Output: 0

# Creazione di istanze della classe
istanza1 = MiaClasse(10)
istanza2 = MiaClasse(20)

# Accesso alla variabile statica tramite un'istanza (è comune
a tutte le istanze)
print(istanza1.variabile_statica)  # Output: 2
print(istanza2.variabile_statica)  # Output: 2
```

Nel codice sopra, `variabile_statica` è una variabile associata alla classe, e non a un'istanza specifica. Può essere acceduta utilizzando sia il nome della classe che il nome di un'istanza.

Metodi Statici:

```
pythonCopy code
class MiaClasse:
    @staticmethod
    def metodo_statico():
        print("Questo è un metodo statico.")

# Chiamata al metodo statico attraverso il nome della classe
MiaClasse.metodo_statico()  # Output: Questo è un metodo stat
ico.
```

```
# Chiamata al metodo statico attraverso un'istanza (anche pos
sibile, ma non è la pratica consigliata)
istanza = MiaClasse()
istanza.metodo_statico()  # Output: Questo è un metodo static
o.
```

@staticmethod

Utilizzato per definire un metodo statico all'interno di una classe. Un metodo statico è associato alla classe anziché a un'istanza specifica e può essere chiamato direttamente dalla classe o dall'istanza.

A differenza dei metodi di istanza, i metodi statici non hanno accesso all'istanza stessa (`self`) e non possono accedere o modificare gli attributi dell'istanza.

```
class MiaClasse:
    variabile_statica = 0

    def __init__(self, valore):
        self.valore = valore
        MiaClasse.variabile_statica += 1

    @staticmethod
    def metodo_statico():
        print("Questo è un metodo statico.")

# Chiamata al metodo statico attraverso il nome della classe
MiaClasse.metodo_statico()  # Output: Questo è un metodo stat
ico.

# Creazione di un'istanza della classe
istanza = MiaClasse(42)

# Anche possibile, ma non è la pratica consigliata
```

```
istanza.metodo_statico()  # Output: Questo è un metodo statico.
```